

Rapport Projet informatique électronique

Auteurs

BEN AMEUR Bilal
KORZENIOWSKI Arthur
BISSAUGE Ludovic
SITTHIVONGS Fabien

Sommaire

1. Introduction.....	3
2. Compilateur.....	4
3. Machine Virtuelle	4
4. Interpréteur Grafcet	7
5. Assembleur	8
6. Partie microcontrôleur	10
7. Présentation du détecteur de métaux	14
8. Etape 1 - Présentation des caractéristiques électroniques.....	15

1. Introduction

Au cours du semestre 3 nous avons étudié le langage de programmation C qui est un langage de bas niveau (relativement au JAVA et au python) permettant de manipuler dynamiquement les espaces mémoire ainsi que les fichiers du système. Ainsi nous avons étudié les notions suivantes :

▣ La compilation séparée ▣ L'algorithmique basique ▣ L'introduction à un langage de programmation (pour mieux comprendre les principes de fonctionnement des langages informatiques) ▣ La manipulation de listes chaînées ▣ La lecture de fichiers au format texte ▣ La manipulation de variables dont la mémoire est dynamiquement allouée.

Au sein du projet du semestre 4 se trouve la création d'un langage de programmation pour modéliser le fonctionnement des Grafcet. L'objectif est de définir notre propre machine virtuelle (qui fonctionnera en langage C) ainsi que son jeu d'instruction, son compilateur et son assembleur dédié.

Le but de ce document est donc de présenter les différents éléments travaillés jusqu'à aujourd'hui pour réaliser ce projet.

Une présentation des instructions et de la machine virtuelle sera faite dans un premier temps. Puis un exemple de modélisation d'un Grafcet en langage C sera développé avant de finir par la description du travail déjà réalisé sur l'assembleur dédié à notre machine virtuelle.

2. Compilateur

Un compilateur est un programme informatique qui transforme un code écrit dans un langage de programmation (compréhensible par l'homme) en un code en langage machine (langage de plus bas niveau) pour qu'il soit ensuite exécuté par une machine. La compilation comprend plusieurs étapes :

- L'analyse lexicale
- L'analyse syntaxique
- La génération du code cible

L'analyse lexicale découpe le code en portion de code dont on va identifier les mots clés et les structures. Cette sous partie de code respecte un langage régulier reconnaissable par un automate à états finis. Cette étape est réalisée par le fichier .lex fourni en début de projet.

L'analyse syntaxique va étudier chaque sous partie du code afin de générer un AST (Abstract Syntax Tree) qui transforme les instructions linéaires en arbres syntaxique. Cette analyse du programme permet de détecter les possibles erreurs de programmation.

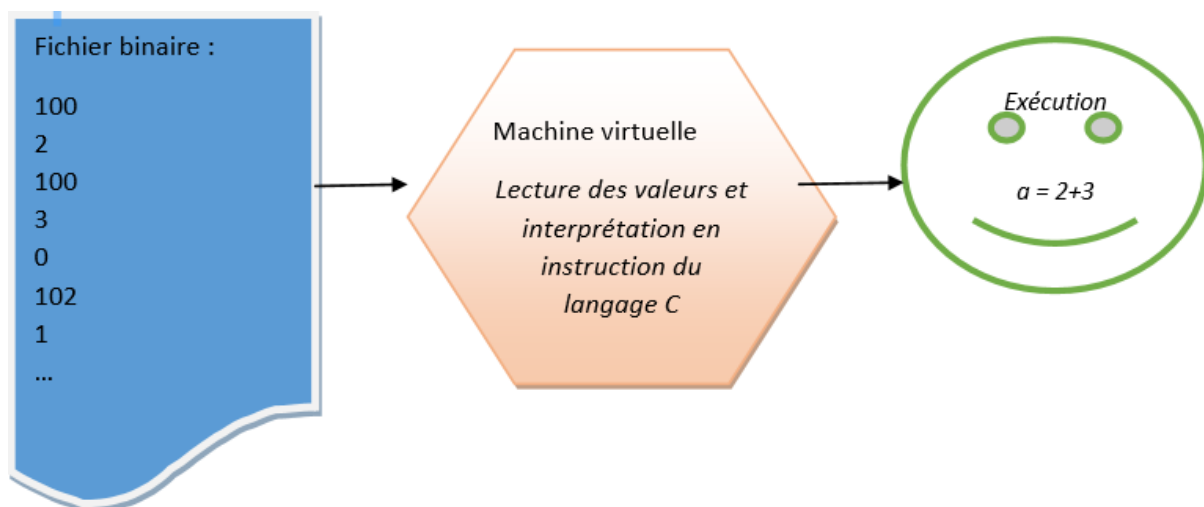
Cette phase est réalisée par le fichier .bison fourni

La génération de code cible va traduire l'AST vers le code machine cible. Ce dernier peut ensuite être interprété par une machine afin d'exécuter le code source.

3. Machine Virtuelle

La machine virtuelle est un programme développé en langage C permettant d'interpréter le langage Grafcet que nous développons pour qu'il puisse être exécuté par notre machine Linux.

Il reçoit en entrée un fichier binaire constitué d'une suite de nombre. Cette suite de nombre est interprétée soit comme des instructions, soit comme des adresses de variables ou encore comme des entiers qui seront des opérandes pour les calculs. Ci-dessous un graphique représentant les le fonctionnement de la machine virtuelle :



Nous y avons défini un jeu d'instructions permettant de réaliser un certain nombre de calculs usuels (addition, soustraction, multiplication) ainsi que des manipulations de mémoire de notre machine virtuelle.

Voici donc les instructions et leur code associé qui nous permettront d'exécuter le code source du Grafcet :

```
#define I_ADD 0
#define I_MULT 1
#define I_SUB 2

#define I_AND 8
#define I_OR 9
#define I_NOT 10
#define I_LT 11
#define I_EQ 12
#define I_GT 13

#define I_INC 14
#define I_DEC 15

#define I_PUSHI 100
#define I_PUSH 101
#define I_POP 102

#define I_JF 200
#define I_J 201

#define I_HALT 400
#define I_END 500
```

Certaines instructions permettent de manipuler deux éléments :

- La pile : tableau de données dans lequel les opérandes des calculs sont ajoutés.
- Le tableau de variable : tableau de données dans lequel les variables sont stockées.

a. Instructions de manipulation de la pile

- **ADD (respectivement SUB):** cette instruction permet de sommer (respectivement soustraire) les deux derniers éléments placés dans la pile puis de stocker dans la pile le résultat
- **MULT :** cette instruction multiplie les deux derniers éléments stockés dans la pile avant d'y stocker le résultat.
- **AND :** cette instruction applique un "&" logique entre les deux dernières valeurs dans la pile. Les deux valeurs doivent être une valeur booléenne (0 ou 1), dans le cas contraire l'instruction ne sera pas réalisée.
- **OR :** cette instruction applique un "|" logique. Les mêmes conditions que pour le AND s'appliquent.
- **NOT :** applique un "!" à la dernière valeur de la pile. Mêmes restrictions que pour le AND.
- **LT (respectivement GT ou EQ) :** vérifie si le dernier élément de la pile est plus grand (respectivement plus petit ou égal) à l'élément qui le précède directement.
- **INC (respectivement DEC) :** incrémente (respectivement décrément) la dernière valeur stockée dans la pile.

- **PUSHI** : pousse dans le premier emplacement libre de la pile la valeur d'un entier spécifié dans le fichier binaire.
- **PUSH** : pousse dans le premier emplacement libre de la pile la valeur de la variable dont l'adresse dans le tableau de variable est spécifiée dans le fichier binaire.
- **POP** : retire la dernière valeur de la pile pour la stockée dans la variable dont l'adresse dans le tableau de variable est spécifié dans le fichier binaire.

b. Instructions modifiant la lecture du fichier

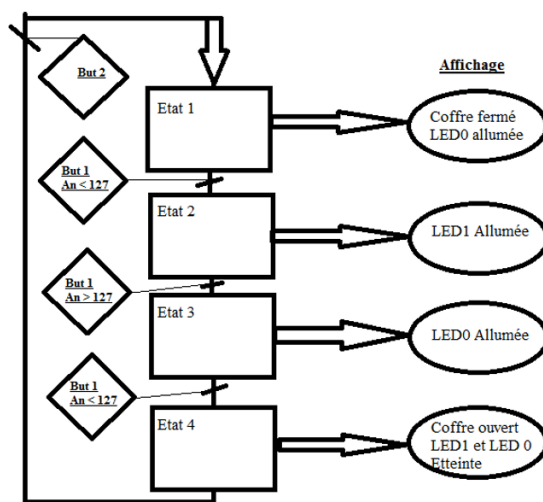
- **JF** : Suite à un test (GT, LT, EQ), si le résultat est faux alors un saut est réalisé à l'étiquette spécifiée dans le fichier binaire. Si le test est correct alors le saut n'est pas réalisé et la suite du code est exécutée.
- **J** : le saut est directement effectué sans aucune condition.

A chacune de ces instructions est associé un code (le numéro précisé dans la figure ci-dessus). C'est ce code qui permet à la machine virtuelle de différencier une instruction d'un opérande ou d'une adresse de variable. L'instruction est réalisée en langage C.

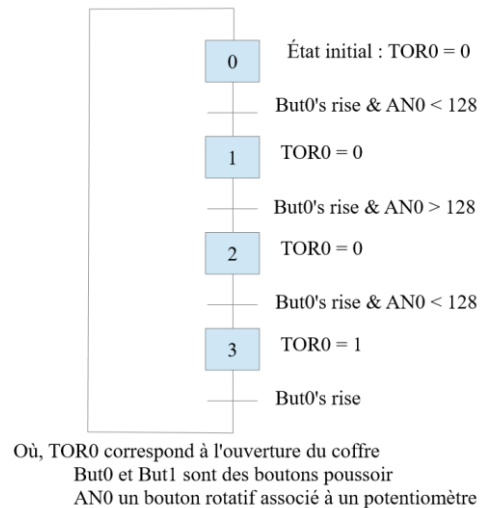
4. Interpréteur Grafcet

Cet interpréteur est la modélisation en langage C d'un Grafcet décrivant le fonctionnement d'un coffre fort.

Cette action correspond aux analyses syntaxique et lexicale. Néanmoins cette étape nous a permis de travailler sur l'écriture d'un automate en langage C. Un grafcet devant respecter plusieurs règles (telles que : un seul état actif à la fois, franchissements des transitions instantanées,...), sa traduction en langage C devient complexe. En effet, un système de stabilité et d'appel-réponse doit être intégré au code grafcet afin de gérer le bon déroulement des états et des transitions. Si on exécute le programme grafcet.c, on peut tester le grafcet d'ouverture du coffre fort. Les paramètres d'entrée (boutons poussoirs et rotatif) doivent être rentrés manuellement par l'opérateur mais on peut suivre l'avancement du grafcet "en temps réel", ce qui nous a permis de vérifier que le programme est opérationnel.



Grafcet synoptique



Grafcet orienté programmation

Le programme est composé d'une boucle while(1) dans laquelle toutes les actions sont effectuées. Cela représente le fait que les états du grafcet sont bouclés. Dans cette boucle infinie le niveau haut et bas des boutons est mis à jour. Puis, tant qu'un état de stabilité n'est pas atteint (tous les états précédents sont égaux aux états actuels) les transitions sont évaluées. Dès que l'état de stabilité est atteint alors on demande le nouvel état des boutons.

Un cycle correspond à une évaluation des transitions, car elles peuvent entraîner un passage entre différents états.

Les états sont représentés par deux variables :

- L'état présent (qui représente l'activité de l'état au cycle actuel)
- L'état précédent (qui représente l'activité de l'état au cycle précédent)

A l'exécution du programme l'état actuel est affiché ainsi que les sorties associés à l'état.

Une erreur a cependant été mise en évidence durant le développement de cette modélisation. En effet, lorsque l'état 3 est actif, que les boutons 1 et 2 sont au niveau haut et que le potentiomètre (An) est inférieur à 128, alors la transition de l'état 3 à l'état 4 est réalisée. Cependant, la transition de l'état 4 à l'état 1 l'est aussi ainsi que celle entre les états 1 et 2. Ce qui fait que les états 4, 1 et 2 deviennent actifs en même temps.

Cela est dû au fait que la modélisation ne prend pas en compte qu'à la fin d'un cycle les boutons sont relâchés. Il est donc nécessaire de forcer leur retour au niveau bas après évaluation de la stabilité.

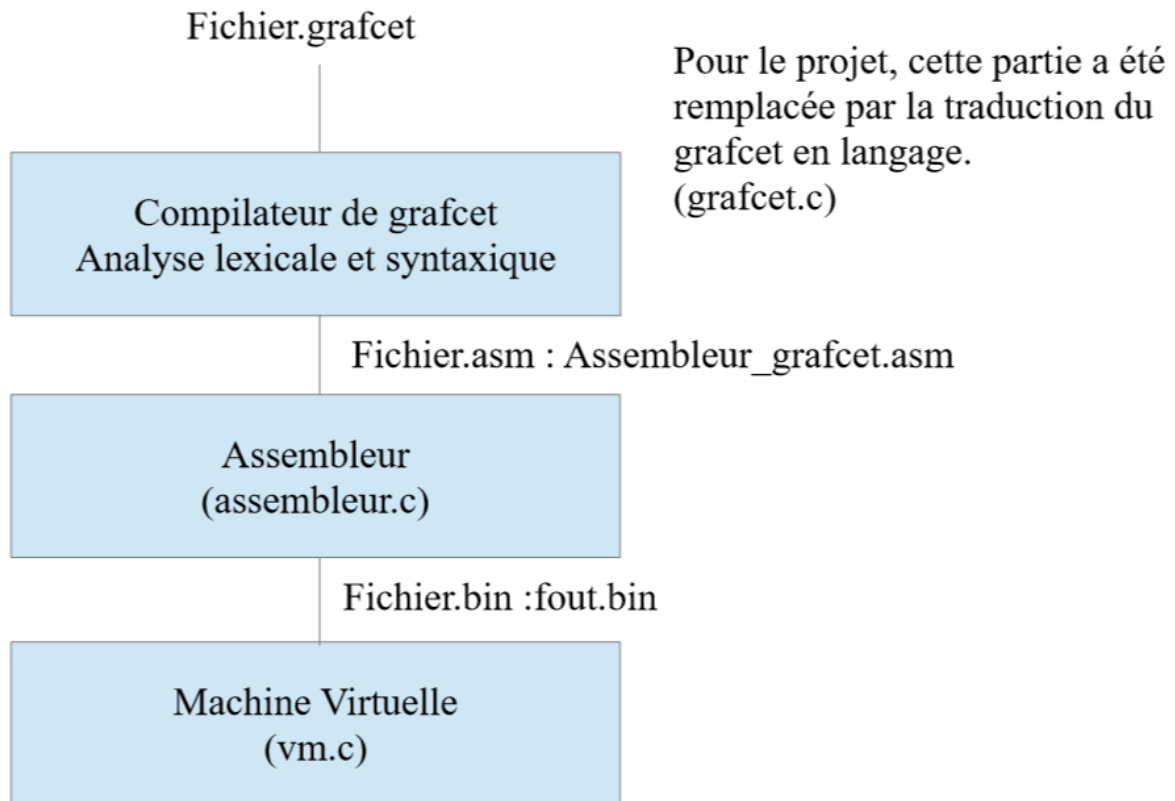
5. Assembleur

Ensuite nous avons traduit, la version du grafcet écrit en langage C vers un code assembleur qui se rapproche du code machine. Ce code transforme toutes les lignes de code (affectations, boucles while, for, conditions if, switch, ...) en instructions simples interprétables par la VM, mais le code assembleur ne peut pas être lu par cette dernière. Il faut donc réaliser une dernière étape avec d'obtenir le code binaire.

Cette dernière est l'assembleur. Ce programme fait la traduction du fichier assembleur vers un code binaire. Le fichier .asm, encore compréhensible par l'homme, permet d'avoir une approche du code binaire. Néanmoins, une machine ne peut pas interpréter ce code du fait qu'il n'y a pas de système d'ordre par indice, ce qui rend les JUMP (qui code les boucles et les structures de conditions IF et SWITCH) impossible à répertorier pour la machine. C'est donc le rôle de l'assembleur. Il agit en trois étapes.

- La première consiste à lire le fichier .asm une première fois ligne par ligne. Pendant cette première lecture, il supprime les éventuels commentaires pour le garder que les instructions utiles. Les instructions sont ensuite interprétées suivant leur type. Les instructions simples, qui ne nécessitent pas d'autres paramètres ou un seul paramètre, sont remplacées par leur code binaire et placées dans un tableau de code (avec leur paramètre). Les instructions plus complexes sont les JUMP et sont traitées en deux étapes. Tout d'abord, on note l'emplacement, dans le code, où chaque JUMP est appelé ainsi que le nom de la référence associée. En parallèle, on note dans un second tableau chaque étiquette (marquée avec un ":") que l'on rencontre ainsi que l'indice de l'instruction où elle est marqué.
- La deuxième phase est une lecture des tableaux d'étiquettes et de références précédemment remplis pour lier les JUMP avec leur étiquette respective. Pour ce faire, parcourt tous le tableau de références et on cherche dans le tableau d'étiquettes le nom qui correspond à la référence. Puis on note l'indice de l'instruction de l'étiquette, dans le tableau du code à l'indice du paramètre du JUMP qui a été mémorisé lors de la première lecture.
- Une fois toutes les références résolues, on possède un tableau d'entier d'instructions qui correspond au code source. Ce code est ensuite écrit dans un fichier de sortie binaire. Ce fichier peut alors être exécuté par la VM.

On obtient donc l'architecture suivante pour notre projet :



Le fichier makefile mis en annexe avec les fichiers ci-dessus permet de générer les exécutables des code assembleur.c et vm.c.

Pour obtenir le fichier binaire, on tapera la commande suivante : `./assembleur
Assembleur_grafcet.asm fout.bin`

Pour faire fonctionner la VM, on tapera : `./vm fout.bin`

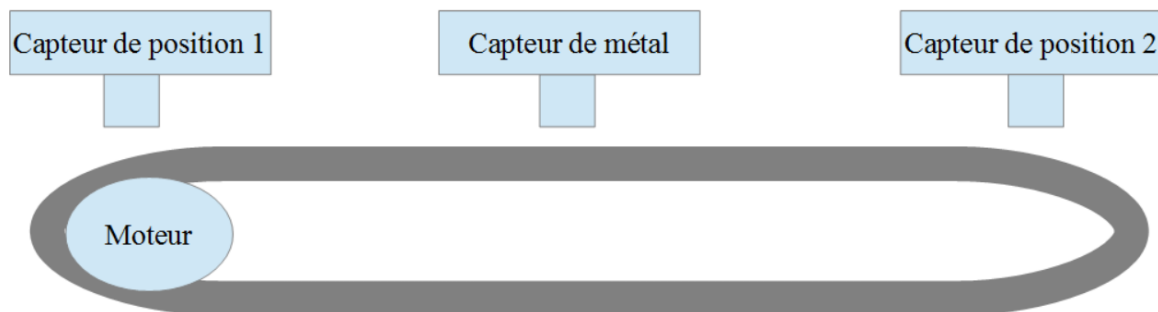
Ici, le code dans fout.bin nécessite des variables qui ne sont pas fournies par la VM, la commande restera donc bloquée (le programme bouclera de manière infinie dans l'une des boucles du grafcet).

Pour tester la VM, nous avons fait un code très simple : `./vm ex1.bin`

6. Partie microcontrôleur

Cette partie s'inscrit dans la continuité du projet de langage C dont l'objectif est de compiler un grafcet afin de générer un fichier binaire. Ce dernier doit être par la suite envoyé sur la carte STM32. La carte équipée de la machine virtuelle faite pendant le projet doit exécuter le code binaire reçu et piloter le coffre fort. Néanmoins, un changement du cahier des charges nous impose de travailler sur une chaîne de production à la place du coffre fort. Les codes anciennement générés pour le coffre fort ne pourront donc pas être réutilisés pour l'application avec micro-contrôleur.

Voici un schéma de la chaîne de production à piloter :



Dans un premier, nous avons déplacé la machine virtuelle faite sous linux dans le micro-contrôleur. Le nombre d'instructions étant restreint, la capacité mémoire de la carte est donc suffisante pour accueillir la VM.

Afin de nous assurer de son bon fonctionnement, nous la testons avec un programme simple

```
code[0] = I_PUSH;
code[1] = 0;                                     // variable[0] : PIN0 soit PC8
code[2] = I_PUSH;
code[3] = 1;                                     // variable[1] : PIN1 soit PC9
code[4] = I_AND;
code[5] = I_POP ;
code[6] = 2;                                     // variable[2] : LED
code[7] = I_HALT;
```

Les instructions I_PUSH 0 et I_PUSH 1 consiste à mettre sur la pile les valeurs des variables qui sont aux indices 0 et 1 dans le tableau de variables. Pour le test, il s'agit de 2 pins initialisées en entrées PC8 et PC9.

Un ET logique va être ensuite effectué et le résultat va être mis dans le tableau variable à l'indice 2. Cette variable correspond ici à la LED (PA5).

Le code est d'abord écrit en brut dans le micro-contrôleur et on effectue le code suivant en permanence :

```
pc = 0;

variable[0] = ((GPIOC -> IDR & 0x0100)>>8); // lecture de la pin0

variable[1] = ((GPIOC -> IDR & 0x0200)>>9); // lecture de la pin1

Run();

if(variable[2] == 0)

GPIOA->ODR &= ~(1<<5); // bit 5 à '0' extinction de la LED

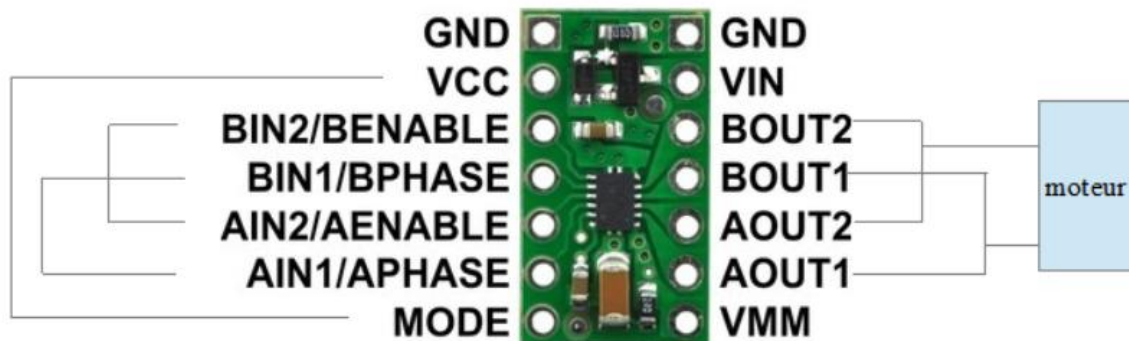
else

GPIOA->ODR |= (1<<5); // bit 5 à '1' allumage de la LED
```

La première étape consiste à lire l'état des deux pins d'entrées et de mettre leur valeur dans le tableau de variable à l'indice correspondant à la variable. Puis on exécute la machine virtuelle, qui va mettre dans variable[2] la résultat du ET logique (ce qui correspond aux instructions I_POP 2 du code binaire). Enfin, on test l'état de cette variable pour l'afficher sur la LED.

Ensuite, nous avons étudié le moteur qui permet de faire bouger le tapis roulant. Il s'agit d'un moteur à tension d'alimentation de 12 V. Le micro-contrôleur ne pouvant fournir assez de courant pour alimenter ce dernier, nous décidons d'utiliser un driver de moteur : le DRV8835. Il permet d'alimenter un moteur en tension continue de 0-11V jusqu'à 1,5A, ce qui nous convient pour le projet.

Schéma du montage du DRV8835 :



Voici son mode de fonctionnement, si l'on active le mode :

Simplified drive/brake operation with MODE=1 (PHASE/ENABLE)				
xPHASE	xENABL E	xOUT 1	xOUT 2	operating mode
0	PWM	PWM	L	forward/brake at speed <i>PWM</i> %
1	PWM	L	PWM	reverse/brake at speed <i>PWM</i> %
X	0	L	L	brake low (outputs shorted to ground)

(source : site du DRV8835)

Pour activer notre chaîne de production, nous devons être capable de faire les 3 actions du tableau ci-dessus.

Il nous faudra donc commander une pin du micro-contrôleur en PWM qui sera connecté aux **ENABLE** et une pin pour la phase qui donnera le sens de fonctionnement du moteur.

Pour la PWM, on utilise la pin PA8 et le timer TIM1, avec un signal de fréquence 100 kHz.

Pour le sens, on utilise la pin PA5, configurée en sortie classique.

Une fois cela effectué, nous avons testé les fonctions qui permettent de faire avancer et reculer le tapis de la chaîne de production. Ce qui nous a permis de certifier le bon fonctionnement du code. Pour ce faire, nous utilisons deux pins configurées en entrée et nous exécutons le code suivant :

```
if(onOffPWM)

    SetPWM(dutyCycle);

else

    ClearPWM();

Sense(senseValue);
```

La constante onOffPWM et senseValue correspondent aux deux pins d'entrée (respectivement PA9 et PC7). Si on active la pin PA9 on peut observer que le tapis de la chaîne de production avance. Si on change l'état de la pin PC7 le tapis recule. Dès lors, nous pouvons considérer que notre code pour diriger le moteur est fonctionnel.

Maintenant que nous avons une machine virtuelle fonctionnel et un driver de moteur opération, il nous faut les lier.

L'objectif étant que la VM va exécuter le programme binaire d'un grafcet. Cette dernière va remplir le tableau de variable suivant le code source et le programme du micro-contrôleur doit ensuite mettre à jour la configuration du moteur (PWM et sens).

De manière générale on souhaite effectuer la boucle suivante :

- Lecture des entrées
- Exécution de la VM
- Écriture des sorties

Comme nous n'avons pas de code binaire de grafcet fonctionnel (c'est à dire avec la gestion des états et transitions ainsi que la gestion de la stabilité du grafcet), nous ne pouvons pas travailler sur la partie exécution de la VM avec un grafcet.

Nous travaillons donc sur les fonctions de communications avec les pins du micro-contrôleur.

La fonction de lecture des entrées permet de récupérer l'état des capteurs de position et de métal, et de place leur valeur dans le tableau de variable à l'indice correspond à chaque capteur. Une fois le Run de la VM lancé, ces variables vont être interprété par la VM puis, en fonction du code source, les variables de sorties vont être inscrites dans le même tableau aux indices correspondant à chaque sortie. Ensuite, une fois la gestion de la stabilité effectuée, la VM redonne la main au programme du micro-contrôleur pour mettre à jour les sorties. Cette

fonction va lire dans le tableau de variable, les valeurs aux indices correspondant à chaque sortie (par exemple la PWM et le sens) afin de piloter les pins connectées au moteur.

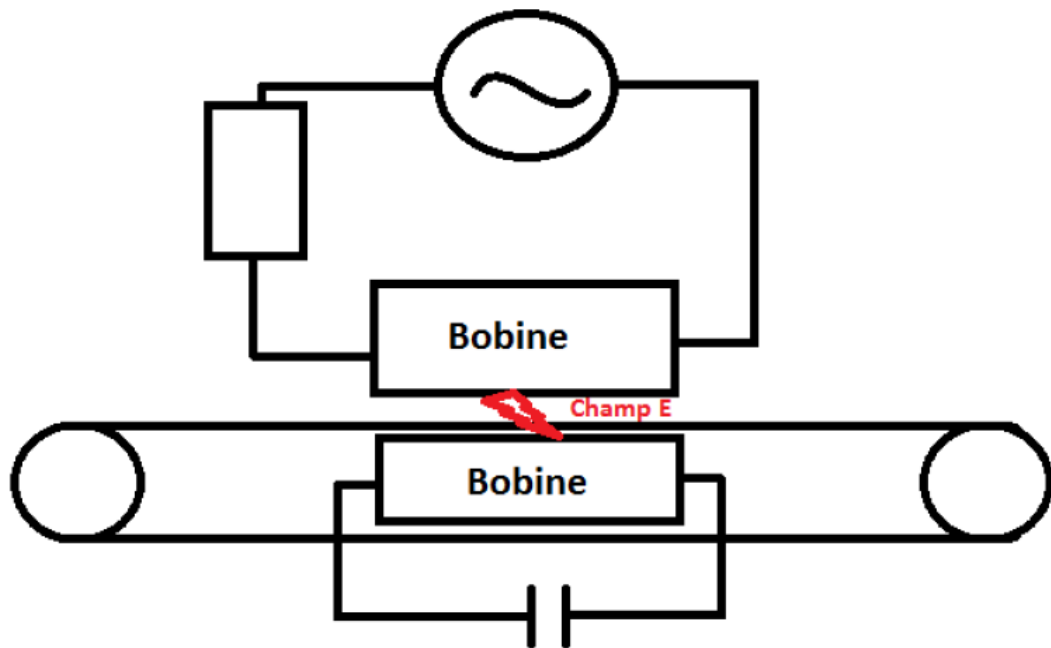
7. Présentation du détecteur de métaux

Dans le cadre du projet Electronique Informatique, nous avons réalisé le montage complet d'un détecteur de métal. Le but de ce détecteur, est de vérifier qu'aucun morceau de métal ne se trouve dans des barquettes transitant sur un tapis roulant de production.

Nous allons donc développer l'ensemble des étapes de cette réalisation et décrire les éléments utilisés dans le montage de détection.

Le principe du détecteur repose sur la perturbation des champs électromagnétiques par des éléments métalliques. Nous avons donc utilisé un montage électronique divisé en deux parties:

1. Une partie dite active : composée d'un générateur, d'une bobine et d'une résistance. Cette partie permet de générer un champ électro-magnétique
2. Une partie passive : Composée d'une bobine et d'une capacité. Cette partie fait office de détecteur de variation du champ électromagnétique.



La partie active nous permet de générer le champ E nécessaire à notre capteur.

La bobine de la partie passive va créer un courant grâce à l'induction faite par le champ E.

Dans la partie passive, la tension générée est à une fréquence de fonctionnement f_0 qui est définie comme :

$$f_0 = \frac{1}{2\pi\sqrt{LC}}$$

En fixant la valeur de la capacité, nous pouvons faire varier cette fréquence en modifiant la valeur de l'inductance de la bobine.

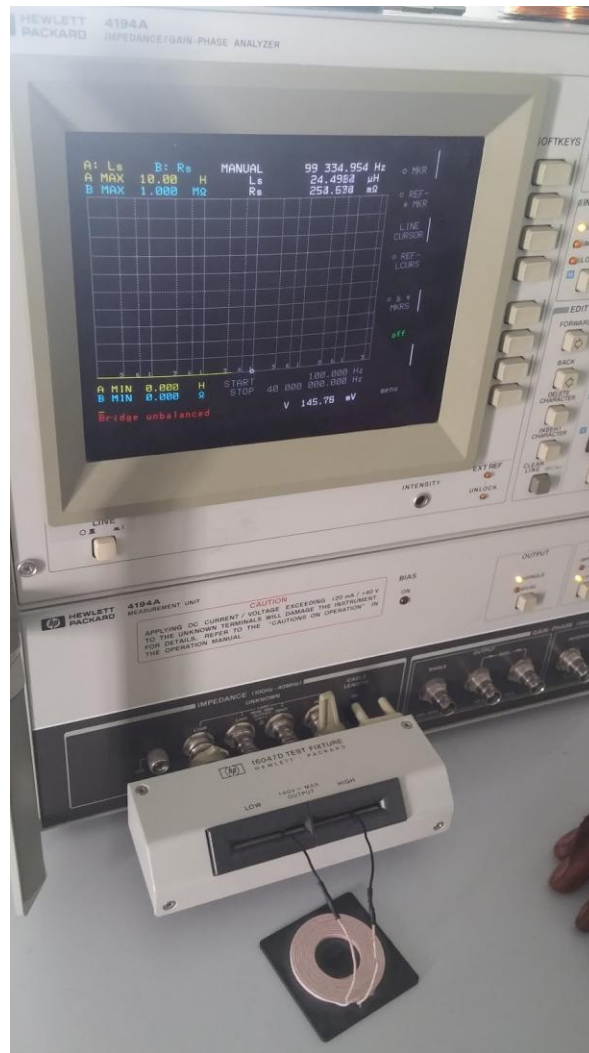
Les variations du champ E n'influencent que la valeur de l'amplitude du signal de sortie du capteur. Donc en connaissant la valeur de la tension lorsque le montage n'est pas en présence d'un objet métallique, nous sommes capable de fixer une valeur étalon de l'amplitude du signal de sortie.

Lorsqu'un objet métallique pénètre le champ magnétique, la tension du signal de sortie chutera. C'est cette différence qui indiquera qu'un métal est présent dans la barquette.

Dans un premier temps nous avons utilisé des bobines fournies par M. DARGA qui sont identiques à celles utilisées dans un autre projet. De plus nous avons ajouté d'autres éléments passifs dans nos montages dont les caractéristiques ont été modifiées au cours de la réalisation du projet afin de corroborer aux exigences du cahier des charges.

8. Etape 1 - Présentation des caractéristiques électroniques

La bobine utilisée dans un premier temps a été une bobine d'inductance : $L = 24\mu\text{H}$ dont la résistance interne était 72 mOhm d'après le constructeur (Des pertes étaient donc à prendre en compte par rapport à ces bobines). Ci-dessous, voici une photographie montrant la caractérisation de ces bobines faites à l'analyseur de réseau:



Cependant cette faible résistance implique que la bobine est perçue dans un montage comme équivalent à un tronçon de circuit fermé. Dans la partie active de notre montage, brancher directement un générateur aux bornes de cette bobine pourrait potentiellement créer un court-circuit, fatal pour notre montage. Il a donc été nécessaire d'ajouter en série à la bobine une résistance de 1 Ohm afin dissiper suffisamment de courant pour éviter le court-circuit.

Du côté de la partie passive, nous n'avons pas la même restriction. En effet la bobine n'est branchée à aucune source connectée à elle. Elle sert justement de source pour la partie passive. Cependant, cette bobine devait être branchée en série avec une capacité permettant de fixer la fréquence de travail.

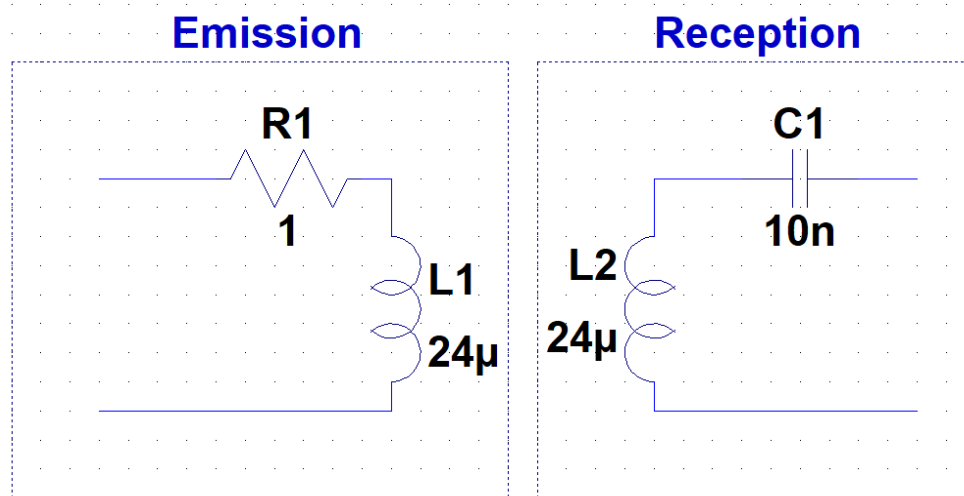
Nous avons défini comme la constante de temps du régime transitoire : $\tau_0 = L \cdot (R + r)$

En choisissant une résistance de 1 Ohm en série avec la bobine on obtient une constante de temps de $2.40 \cdot 10^{-5}$ secondes. Ce temps est négligeable pour notre application et permet d'obtenir un signal exploitable rapidement.

$$f_0 = \frac{1}{2 \cdot \pi \cdot \sqrt{24 \cdot 10^{-6} \cdot 10 \cdot 10^{-9}}} = 324,8737 \text{ KHz}$$

Nous avons choisi une capacité de 10nF ce qui nous donne une fréquence de 324.87 kHz.

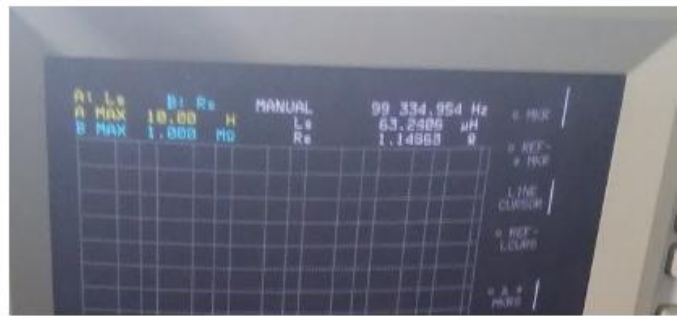
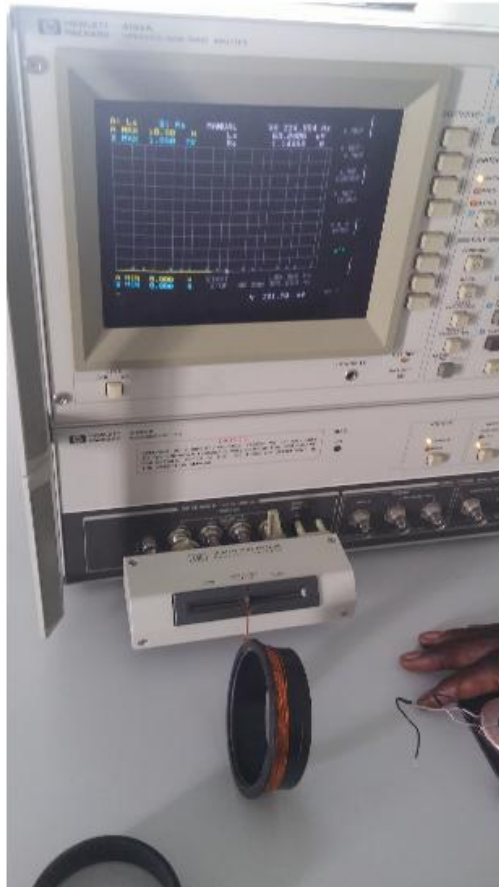
A présent, nous avons la totalité des informations théorique déduites des fiches techniques.
Les schémas d'envoi est le suivant:



A la réception nous avons un montage LC qui va travailler à sa fréquence de résonance afin d'avoir le gain maximal. Nous avons déjà cette fréquence qui est calculée au dessus mais nous pouvons également la mesurer expérimentalement avec un GBF. Il nous suffit d'envoyer un sinus en entrée et de vérifier le sinus que l'on reçoit. Une simple variation de la fréquence permettra d'arriver à une amplitude maximale. Avoir un maximum d'amplitude à la réception, nous permettras d'écarter au mieux les bobines selon la largeur de la chaîne de production

Cependant en soudant le circuit électrique nous nous sommes aperçus que la distance maximale entre les deux bobines était limitée à 2 cm. Cette distance trop courte empêche de placer notre détecteur autour de la chaîne de production sur laquelle nous devons détecter la présence éventuelle de métal dans les barquettes de nourriture.

C'est pourquoi nous avons décidé de réaliser nous-mêmes nos propres bobines afin de satisfaire le cahier des charges. Nous avons donc enroulé du fil de cuivre vernis n fois. Grâce aux cours d'électronique de puissance et à l'expérience, nous savons que le nombres de spires (tours de bobine) influe sur la tension de réception, c'est à dire que si l'on à moins de spires à l'envoi qu'à la réception, il y aura un facteur d'amplification, ce qui pourra nous permettre d'écarter les bobines de réception et de détection du métal sur une induction afin de détecter des éléments métalliques. Nous souhaitons également que la bobine ait un plus grand diamètre afin que la surface couverte par le capteur soit plus grande. A cet effet nous avons récupéré des cercles en plastique que nous avons bobiné afin que la forme de nos bobine reste la même durant les tests.



Du fait que les deux bobines utilisées ont été modifiées, nous avons dû modifier la fréquence de travail du montage. Pour ce faire nous avons réutilisé l'équation précédemment citée:

$$f_0 = \frac{1}{2\pi\sqrt{LC}}$$

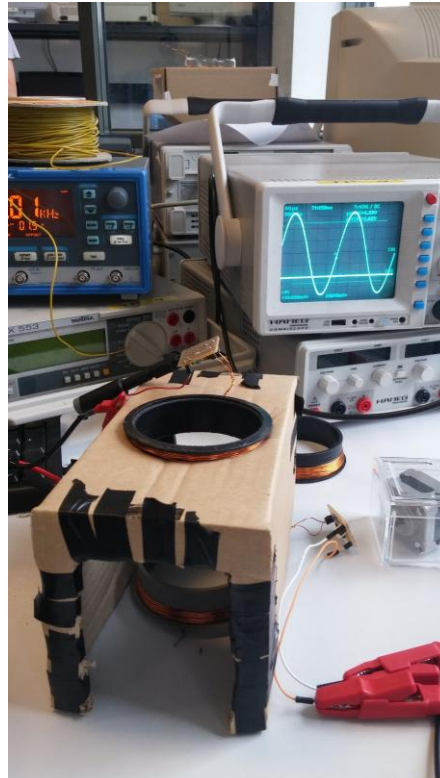
Nous pouvons donc en déduire la valeur de la fréquence de travail qui est à présent de :

$$f_0 = 200.135 \text{ kHz}$$

(ce calcul a pu être corrélé grâce aux mesures réalisées qui nous a donnée une valeur quasi équivalente expérimentalement)

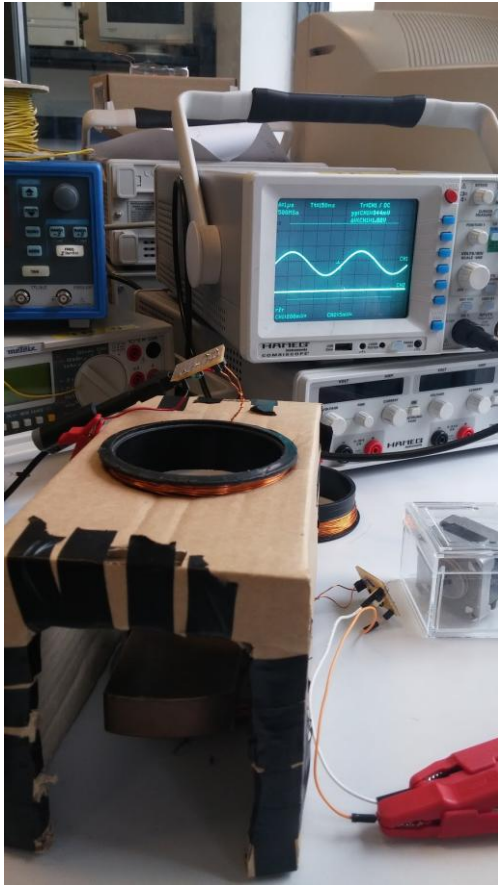
Après avoir monté le système, nous nous sommes rendus compte que lorsque l'on envoyait notre sinus à 3,3 V et la fréquence de résonance, le signal reçu avec un écartement de 1cm correspondait à un sinus de même fréquence mais de tension peak to peak égale à 10,5V. Nous pouvions donc écarter les bobines sans avoir besoin d'amplifier le signal de réception.

Voici le montage finalisé :



On peut donc bien voir la bobine en haut qui fait l'émission et la bobine en bas à la réception. Nous avons également décidé de fabriquer une structure qui permettrait de tenir nos bobines avec un écran fixe afin de ne pas fausser nos mesures. Cette structure prévoit aussi le passage du rail qui transportera la nourriture à tester.

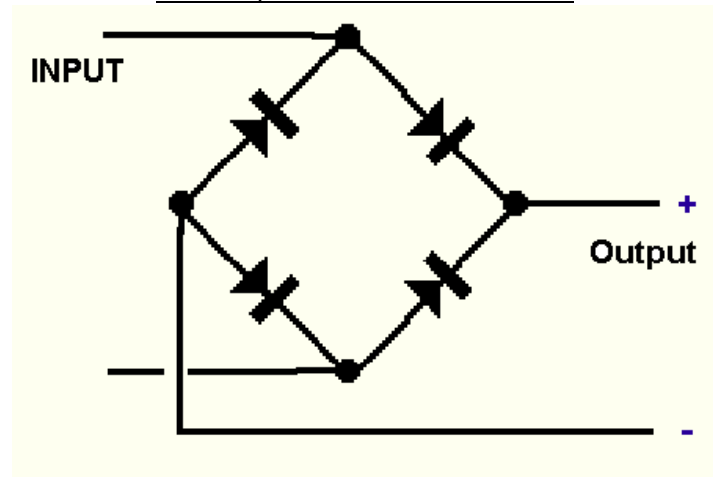
Il nous faut donc maintenant vérifier les influences du passage d'un composants métallique dans le champ électromagnétique qui circule entre les deux bobines :



Nous observons ici, que si l'on place une plaque métallique non loin de la bobine de réception, l'amplitude du sinus s'effondre. Ce qui valide le fonctionnement de notre détecteur de métaux à "moyenne portée". Après un long moment d'étude, nous avons remarqué que le signal de sortie était plus impacté lorsque le composant métallique se situait plus proche de l'antenne de réception que d'émission. Ainsi nous avons modifié le positionnement afin de placer la bobine de réception plus près de la source à analyser, donc en haut de la structure.

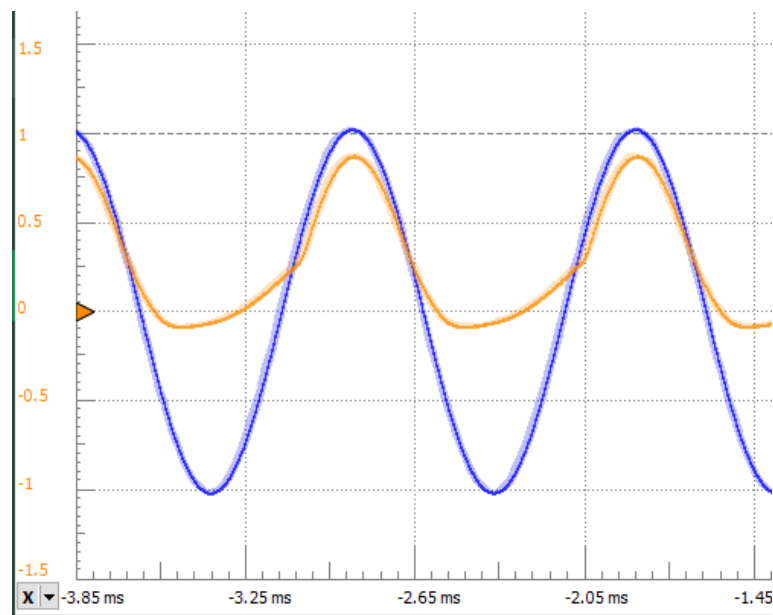
Une fois ce montage fonctionnel et optimisé, notre souhait dans le future serait d'acquérir ce signal avec un micro-contrôleur et donc le sinus rajoute de la complexité inutile dans la réalisation, nous décidons donc de réaliser un pont redresseur afin d'avoir uniquement la variation de tension et ainsi avoir un signal continue qui varie si il y a présence de métal.

Schéma pont diode ou redresseur :



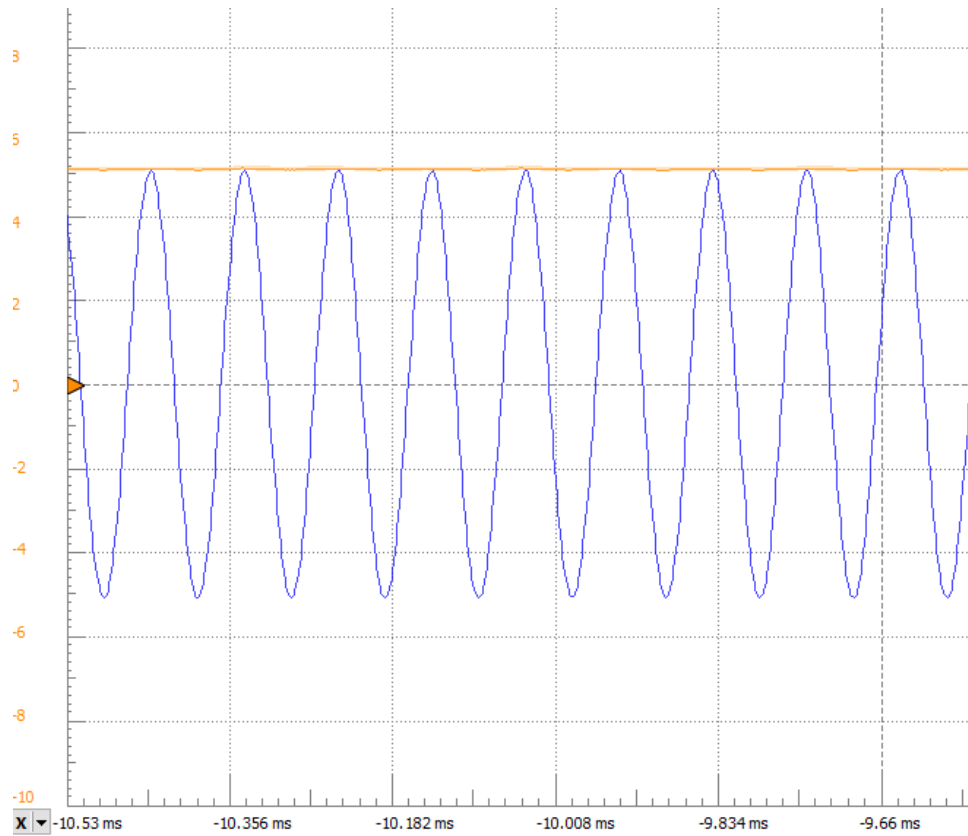
C'est en suivant ce schéma que nous avons réalisé le pont diode. L'inconvénient est que nous avons seulement à disposition des diodes de puissance qui donc ont une tension de seuil élevé, ce qui nous pose problème car on récupère un signal assez faible de l'ordre de 0.5 V or si nous avions eu des diodes avec une tension de seuil moins élevé nous aurions pu avoir un signal DC au environ de 1V qui s'atténue plus et donc avoir une plus grand plage de mesure à analyser.

Mesure entrée et sortie du pont diode :



Nous pouvons voir le fonctionnement typique d'un pont diode. Nous souhaitons lisser ce signal afin d'avoir un signal pseudo continu. Nous rajoutons donc en sortie du montage un condensateur de 1 μ F.

Nous obtenons les tracées suivants avec le condensateur de 1 μ F :



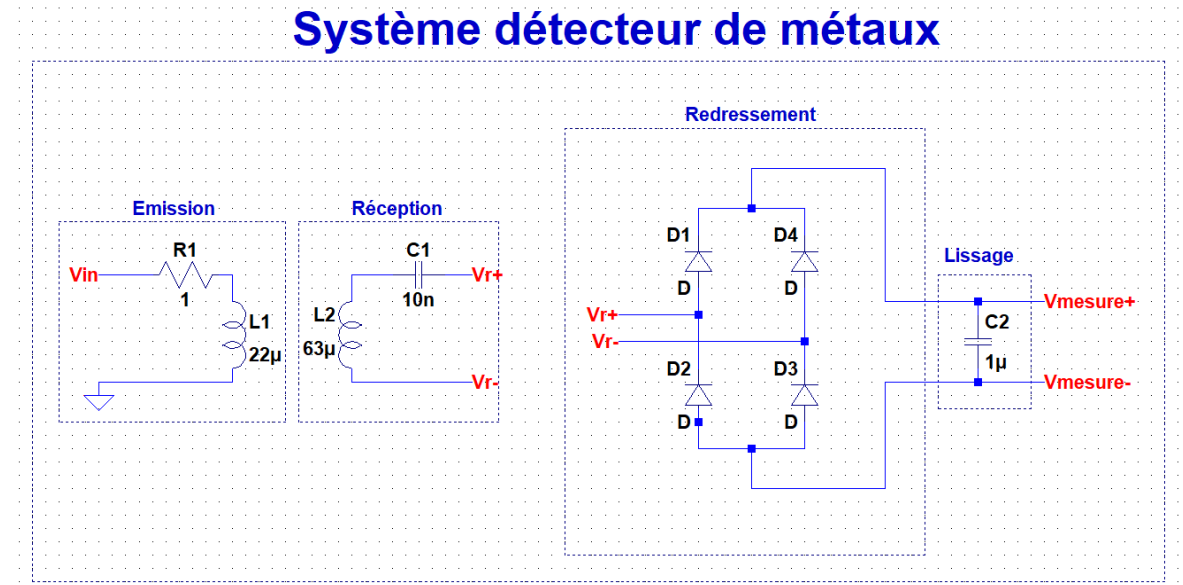
L'étape suivante aurait été d'utiliser l'ADC déjà présent sur le microcontrôleur STM32-F072. Il permet de recevoir en entrée un signal analogique et de le convertir en signal numérique analysable depuis le microcontrôleur. Pour utiliser l'ADC on peut utiliser la pin Vbat qui est plus adapté à l'analyse de notre tension de sortie.

La pin du même nom que la fonction Vbat est située sur la pin 33 de la carte. Cette fonction est destinée à recevoir en entrée un signal d'une batterie et donne en sortie un signal numérique qui est la tension de la batterie. Comme notre tension de sortie est un signal continu, l'utilisation du Vbat se prête à l'analyse de la tension en sortie de notre montage. Il nous suffirait donc de comparer la tension donnée par la pin Vbat à la valeur de référence de la tension en l'absence d'objet métallique.

D'autre part, le constructeur met en place une sécurité concernant la tension d'entrée. Afin d'éviter que la tension continue d'entrée dépasse la tension d'alimentation de l'ADC, un diviseur de tension est placé en entrée de l'ADC. Ainsi la tension convertie est la moitié de la véritable tension. Il aurait donc été nécessaire d'en tenir compte dans la cas d'une comparaison du signal.

Schémas du système :

Systeme detecteur de métaux



Nous avons donc ce schéma final, sur la partie réception nous sommes en masse flottante, il n'y a donc pas de GND.

Sources du projet:

- http://users.skynet.be/on5hq/aide_memoire/couplage_entre_deux_bobines.pdf